DESIGN PATTERNS · CREATIONAL | GoF · 03 / 23 | 4-dive study card

Factory Method

Define a method for creating an object, but let subclasses decide which concrete class to instantiate.

DIVE 01 · 10 km — **Airplane porthole** — The big picture & contours → DIVE 02 · 1 km — **Train window** — Theses, examples, terms → DIVE 03 · 100 m — **Park walk** — How the pieces relate → DIVE 04 · 1 cm — **Microscope** — Exact wording & gotchas

Airplane porthole — DIVE 01 · ORIENTATION — ▼ 10 km

Factory Method defers the choice of class — to a subclass, so a parent’s algorithm can create objects it never names.

— THE CONTOURS

- **Subclass decides the class** — the parent calls a method; children pick the product
- **Creation via inheritance** — override one method, not the whole algorithm
- **One product at a time** — — a whole family is Abstract Factory’s job

— USE IT WHEN — GoF applicability

- **Class can’t anticipate** — a class can’t know the class of the objects it must create ahead of time
- **Subclasses should choose** — you want subclasses to specify which object the parent creates

Creational family: Factory Method · Abstract Factory · Builder · Prototype · Singleton — Factory Method makes one product through subclassing; Abstract Factory composes several of them into a family.

DIVE DEEPER

Train window — DIVE 02 · KEY OBJECTS — ▼ 1 km

— THE THREE PARTS

- `Creator` — declares factoryMethod(); often calls it inside an algorithm
- `ConcreteCreator` — overrides factoryMethod() to return a ConcreteProduct
- `Product` — the interface the Creator returns and the caller uses

— IN THE WILD

- `Collection.iterator()` — each collection returns its own Iterator
- `Application.createDocument()` — GoF’s example — a subclass picks the Document
- `URLStreamHandler` — openConnection() varies per protocol subclass
- `useFactory providers` — Angular / Nest decide the instance at DI time

```
<> Creator.someOperation() → calls this.factoryMethod() → ConcreteCreator returns a ConcreteProduct.
```

> **GoF intent:** “Define an interface for creating an object, but let subclasses decide which class to instantiate.”

DIVE DEEPER

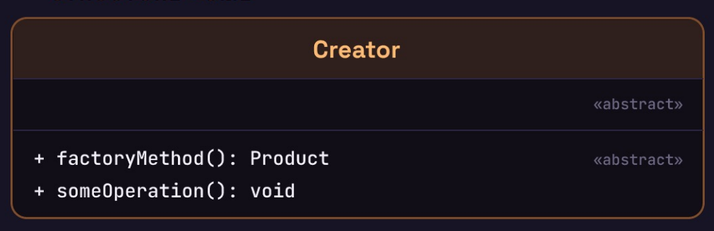
Park walk — DIVE 03 · CONNECTIONS — ▼ 100 m

Creator (owns an algorithm) → `factoryMethod()` (creation hook) → `which subclass?` (the override decides) → ✓ **ConcreteProduct** (returned to all)

— WHY IT FOLLOWS

- → **Parent calls the hook ⇒** the creation step is a method call, not a hard-coded new
- → **Subclass overrides it ⇒** each ConcreteCreator returns its own concrete product
- → **Parent codes to Product ⇒** the surrounding algorithm never names a concrete class

— STRUCTURE · UML



someOperation() calls factoryMethod(); a ConcreteCreator overrides it to return a ConcreteProduct. Clients only ever see the Product interface.

— WHAT IT BUYS YOU

- ✓ **Removes hard-coded classes —** code depends on the Product interface, not concretes
- ✓ **Hooks for subclasses —** gives subclasses one place to alter what gets created
- ✓ **Connects parallel hierarchies —** pairs each Creator subclass with a Product subclass
- ✓ **Allows a default product —** the Creator can supply a default — overriding is optional
- ✓ **Costs a subclass —** you may subclass the Creator just to make one product

Abstract Factory is built from Factory Methods; Template Method usually calls one; Prototype is the alternative when you’d rather clone than subclass.

DIVE DEEPER

Microscope — DIVE 04 · DETAILS — ▼ 1 cm

> **Definition:** “Define an interface for creating an object, but let subclasses decide which class to instantiate; Factory Method lets a class defer instantiation to subclasses.”
> — GoF, “Design Patterns”, 1994

— THE CODE

```
abstract class Dialog {
abstract createButton(): Button;
render() {
const b = this.createButton();
b.onClick(() => this.close());
}
}
class WinDialog extends Dialog {
createButton() { return new WinButton(); }
}
```

— VARIANTS

- **Abstract method** — GoF classic — the subclass must override createX()
- **Default + override** — Creator returns a default; a subclass may replace it
- **Parameterized** — one create(kind) switches on an argument — fewer subclasses
- **Static factory method** — Type.of() / valueOf() — same name, but not GoF’s pattern
- **Template-Method pair** — a creation step inside a fixed algorithm
- **Functional** — pass a factory fn instead of subclassing (JS/TS)

— COMPARE THE ALTERNATIVES

Property	Factory M.	Abstract F.	Builder	Static fac.
Polymorphic (subclass)	✓	~	×	×
Creates a whole family	×	✓	×	×
Hides concrete class	✓	✓	✓	✓
Relies on inheritance	✓	×	×	×
One method, one product	✓	×	~	✓

Best fit → Factory Method: subclass picks one product · Abstract Factory: matching families · Builder: stepwise build · Static factory: named constr
uctor. (“~” = partial.)

— NUANCES & GOTCHAS

- **vs Abstract Factory** — one product via inheritance vs a whole family via composition
- **vs simple/static factory** — a static helper named create() is not GoF’s polymorphic Factory Method
- **Subclass-per-product cost** — you may add a Creator subclass solely to pick a product
- **Parallel hierarchies** — Creators and Products tend to evolve in lockstep
- **Provide a default** — a sensible default product spares every subclass an override
- **JS shortcut** — a function parameter often replaces the whole subclass hierarchy

— INTERVIEW PITFALLS

Q “Factory Method vs Abstract Factory?”
Factory Method makes one product and varies it by subclassing; Abstract Factory makes a family and varies it by swapping a factory object.

Q “Is a static create() a Factory Method?”
Not in GoF terms — that’s a static/simple factory. The real pattern is polymorphic: a subclass overrides it.

Q “Why not just call new?”
So the base algorithm never names a concrete class; subclasses decide what’s built without touching the parent.

Q “What’s the main cost?”
You may need one Creator subclass per product, creating parallel class hierarchies.

— WATCH OUT

⚠ **Anti-pattern:** Don’t subclass just to swap a constructor — with no shared algorithm around the creation, a plain function, an argument, or DI is simpler than a Creator hierarchy.

⚡ **Modern take:** in JS/TS pass a factory function or use DI to choose the implementation; keep real Factory Method subclassing for when a base class owns an algorithm around the creation step.